

TP13 : décomposition en valeurs singulières

ZFAI

1 Introduction

L'objectif de ce TP est de présenter différentes applications de la décomposition en valeurs singulières. On utilisera les bibliothèques `numpy` (pour l'analyse numérique), `PIL` (pour la lecture d'image) et `matplotlib` (pour l'affichage).

Dans la suite, ces trois bibliothèques sont supposées importées.

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

2 Prise en main avec numpy

Pour manipuler des matrices ou des tableaux, on utilise la bibliothèque `numpy`. Dans cette section, on la suppose importée sous l'alias `np`.

Celle-ci contient par défaut des structures de données, des fonctions et méthodes qui facilitent la manipulation de matrices.

Une matrice est représentée en `numpy` par la structure `array`. Par exemple, si on veut représenter la matrice

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & -1 & 2 \end{pmatrix}$$

Il suffit d'écrire :

```
A = np.array([[1,2,3],[0,-1,2]])
```

La manipulation des images se fait également par la manipulation des `array`. Elles sont représentées par défaut par des `array` de triplets d'entiers sur 8 bits. Pour le TP, on les convertit d'abord en `float`.

```
# Charger l'image couleur
img = Image.open("nom_photo.jpg")
A = np.array(img, dtype=float) / 255.0 # Shape: (H, W, 3)
```

transforme le fichier image ayant comme nom `nom_photo.jpg` en `array numpy` avec comme forme `(H,W,3)`, `H` correspondant au nombre de lignes de l'image, `W` au nombre de colonnes, et `3` au nombre de couleurs. La division par 255 permet de ramener chaque coefficient dans l'intervalle `[0,1]` et `plt.imshow` permet d'afficher les images qui sont représentées par des matrices `numpy` dont les coefficients sont compris entre 0 et 1.

Il est possible de spécifier le type d'entrée de la matrice. Pour ce TP, on utilisera possiblement les types suivants :

- `float` (utile pour manipuler des nombres réels)
- `int` (pour les entiers)

Pour préciser le type, on écrit :

```
A = np.array([[0.1,2.5,1],[1.3,2.0,1.1]], dtype = float)
```

On obtient ici une matrice de `float`.

Le type `array` permet de faire différentes opérations matricielles qui sont pratiques. Il est possible de sommer avec `+` et de multiplier avec `@`. Par exemple :

```
A = np.array([[0.1,2.5,1],[1.3,2.0,1.1]], dtype = float)
B = np.array([[1,2],[3,2]], dtype = float)
A+A
B@A
```

permet de calculer `A+A` et `B.A`.

L'algorithme que l'on utilisera par la suite est le suivant :

Entrées :

- A une matrice rectangulaire $m \times n$
- k un entier petit devant le rang de A

Sorties :

un triplet U_k, S_k, V_k où

- U_k est une matrice $m \times k$
 - S_k est une ligne à k réels strictement positifs
 - V_k est une matrice $k \times n$
-
- Calculer la décomposition en valeur singulières de A :
 $A = U S V^t$
 - garder les k premières colonnes de U pour U_k
 - garder les k premières valeurs (les plus grandes) de la matrice diagonale S pour S_k
 - garder les k premières lignes de V^t pour V_k
 - renvoyer U_k, S_k, V_k

La décomposition en valeurs singulières avec `numpy`, se fait à l'aide de la fonction `linalg.svd`. Par exemple

```
A = np.array([[1,2,3],[0,1,2]], dtype = float)
U,S,Vt = np.linalg.svd(A,full_matrices = False)
print(U@np.diag(S)@Vt)
```

permet de calculer U, S, V^t où $U \cdot \text{diag}(S) \cdot V^t$ est égale à la matrice A .

Dans la décomposition donnée par `numpy`, S correspond à un vecteur ligne et V^t est déjà la matrice qui correspond à la transposée de V . Si on a un vecteur ligne S , la fonction `np.diag` permet de construire une matrice dont la diagonale est donnée par S . Si A est de taille $m \times n$, l'option `full_matrices = False` permet d'avoir U de taille $m \times r$, S comme une ligne de réels strictement positifs de taille r et V^t de taille $r \times n$, r désignant le rang de A . Ainsi, dans U et V^t on ne garde que les lignes utiles pour reconstituer A et on a $A = U \cdot \text{diag}(S) \cdot V^t$.

Exercice 1 (Algorithme de compression). Écrire une fonction `compression_rang(A,k)` prenant en arguments une matrice de réels et un entier k et qui renvoie un triplet U, S, V^t construit comme précisé dans l'algorithme

Exercice 2 (Algorithme de reconstitution). Écrire une fonction `reconstitution(U,S,Vt)` prenant en arguments U une matrice rectangulaire de taille $m \times k$, S une matrice ligne ayant k éléments et V^t une matrice rectangulaire de taille $k \times n$ et qui renvoie $U \cdot \text{diag}(S) \cdot V^t$.

3 Application : compression d'images

Exercice 3 (Lecture et transformation d'un fichier en tableau). Écrire une fonction `conversion_image_tableau(nom_fichier)` prenant en argument une chaîne de caractères qui correspond au nom d'un fichier image et qui renvoie le tableau `numpy` représentant cette image.

Exercice 4 (Compression d'une image). 1. Écrire une fonction `compression(A,k)` prenant en argument un tableau `numpy` représentant une image et qui renvoie une liste de trois triplets de la forme

$$[(U_{k_r}, sk_r, Vtk_r), (U_{k_v}, sk_v, Vtk_v), (U_{k_b}, sk_b, Vtk_b)]$$

où le premier (respectivement deuxième, troisième) triplet est une compression par *SVD* de la composante en rouge (respectivement vert, bleu) de A .

2. (★) Dans le cas où A est de taille $m \times n \times 3$, quel est le taux de compression obtenu ?

Exercice 5 (Reconstitution d'une image). Écrire une fonction `reconstitution_image(liste)` prenant en argument une liste de triplets issue d'une compression d'image et qui renvoie un tableau `numpy` représentant l'image compressée. On veillera à ramener chaque coefficient de la matrice dans l'intervalle $[0, 1]$. On pourra remplacer un coefficient négatif par 0 et remplacer un coefficient plus grand que 1 par 1.

Exercice 6 (Tests des fonctions). Écrire un programme qui effectue les opérations suivantes :

- transformation d'une image récupérée sur DingTalk en matrice ;
- compression de cette matrice avec $k = 50$;
- reconstitution de l'image compressée ;
- affichage de l'image compressée.

4 Application : traitement du signal

Exercice 7 (Construction d'une somme de sinusoides). 1. Écrire une fonction `signal(L,M,t)` prenant en arguments deux listes L, M de même taille n et un réel t et qui renvoie le réel

$$\sum_{i=0}^{n-1} M[i] \sin(L[i]t).$$

On rappelle que la fonction `sin` est donnée par `np.sin`.

2. En déduire une fonction `echantillon(L,M,nombre,debut,delta)` prenant en arguments deux listes de réels L, M de même taille, un entier `nombre` correspondant à la taille de l'échantillon du signal un réel `debut` qui correspond au premier temps de l'échantillon et un réel `delta` qui correspond à l'intervalle de temps entre deux échantillons successifs et qui renvoie la liste des valeurs du signal échantillonné. Autrement dit, si on note S le signal représenté par L et M , on renvoie la liste

$$[S(t), t \in \{debut, debut + delta, \dots, debut + (nombre - 1) * delta\}]$$

3. La bibliothèque `random` contient différentes fonctions permettant de simuler des variables aléatoires. En particulier, `random.gauss` est une variable aléatoire suivant une loi normale $\mathcal{N}(0, 1)$.

De la même façon, écrire une fonction `echantillon_bruit(L,M,nombre,debut,delta)` qui renvoie la liste

$$[S(t) + \varepsilon(t), t \in \{debut, debut + delta, \dots, debut + (nombre - 1) * delta\}]$$

$\varepsilon(t)$ étant une variable aléatoire suivant une loi normale $\mathcal{N}(0, 1)$. On suppose de plus que les $\varepsilon(t)$ sont mutuellement indépendantes.

Exercice 8 (Construction d'une matrice de Hankel). Étant donnée une liste de réels $L = [s_0, \dots, s_{n-1}]$, un nombre de colonnes N , la matrice de Hankel correspondante est donnée par :

$$H = \begin{pmatrix} s_0 & s_1 & s_2 & \cdots & s_{N-2} & s_{N-1} \\ s_1 & s_2 & \cdots & \cdots & s_{N-1} & s_N \\ \vdots & \vdots & & & & \vdots \\ \vdots & \vdots & & & & \vdots \\ s_{n-N-1} & s_{n-N} & \cdots & s_{n-3} & s_{n-2} \\ s_{n-N} & s_{n-N+1} & \cdots & s_{n-2} & s_{n-1} \end{pmatrix}$$

Écrire une fonction `hankel(L,N)` prenant en arguments une liste de réels, un nombre de colonnes N (plus petit que le nombre de valeurs) et qui renvoie un `array numpy` représentant la matrice de Hankel correspondante.

Exercice 9 (Algorithme de Cazdow). On rappelle l'algorithme de Cazdow. Étant donné un échantillon L contenant du bruit, un nombre de colonnes N , un seuil k , on répète un certain nombre de fois les étapes suivantes :

- calcul de la matrice de Hankel associée à L avec N colonnes,
 - calcul de décomposition en valeurs singulières en gardant uniquement les k plus grandes valeurs,
 - calcul de reconstitution de cette décomposition en valeurs singulières tronquées
 - calcul du nouvel échantillon L , obtenu en moyennant chaque anti-diagonale.
1. Écrire une fonction `un_cazdow(L,N,k)` prenant en argument un échantillon L , un nombre de colonnes N et un seuil k et qui renvoie un nouvel échantillon obtenu par une étape de l'algorithme de Cazdow.
2. En déduire une fonction `itere_cazdow(L,N,k,nb)` qui renvoie un échantillon obtenu en appliquant `nb` fois l'algorithme de Cazdow.

Exercice 10 (Tests de reconstruction par l'algorithme de Cazdow). On utilisera essentiellement les fonctions implémentées précédemment.

Écrire un programme qui effectue les opérations suivantes :

1. construction d'un échantillon L de taille 101 de la fonction $x \mapsto 5 \sin(2x) + 2 \sin(3x) + 8 \sin(5x)$ qui commence à $t = 0$ et qui finit à $t = 1$;
2. construction du même échantillon M mais bruité par une gaussienne ;
3. construction d'un échantillon débruité E en appliquant de l'algorithme de Cazdow avec M , $N = 30$, $k = 3$, $nb = 100$.
4. Affichage de L, M, E sur le même graphe avec des couleurs différentes.